```
void blowBubbles() {
  for (int i=0; i<3; i++) {
    lcd.setCursor(x, y);
    lcd.write(i);
    delay(time);
  }
  lcd.clear();
}
```

This sketch uses many of the LiquidCrystal functions that we have already discussed, but it also adds a couple new ones. Let's look at these two functions before moving on to our next library.

scrollDisplayLeft() and scrollDisplayRight()

Each time these functions are called, the display will be shifted one character to the left or right, depending on which function we call. We placed these functions in `for` loops with a delay and a counter to track the horizontal position of the cursor. Take for example the following:

```
for (int i=0; i<steps; i++) {
  lcd.scrollDisplayRight();
  delay(time);
  x++;
}
```

This loop will call the function `scrollDisplayRight()` once each time through the loop for the total number of steps passed to the function. The loop will also delay for the specified amount of time and increment the horizontal position by one.

That might seem like a pretty exhaustive look at the LiquidCrystal library, but believe it or not there are even more functions to do all sorts of things that we haven't even mentioned here. However, rather than dwelling too long on this one library, let's keep moving to some of the other interesting libraries that we need to talk about.

Servo Library

Moving things around on a screen is pretty cool, but let's now look at how we can physically move material things around. The best way to move something with the Arduino is to use a motor. We already looked at a simple motor with the DC fan in Project 4, which is functionally equivalent to controlling any simple DC motor. Two other common types of motors include the hobby servo and the stepper motor. Both of these motors can be difficult to work with without the awesome libraries that are a part of the Arduino platform. The first motor that we'll discuss, the hobby servo, typically moves in 180º arcs. While initially used in radio-controlled hobby applications, the servo has quite the following in robotics and animatronics because of its integrated electronics, gearing, and positional feedback, which allow for fairly precise control.

Typically, to control the servo, a signal of varying pulse widths is sent to the servo over a regular time period and the servo moves to the position that corresponds to that pulse. As long as the signal is refreshed about every 20 milliseconds, the servo will maintain its position. On our Hitec servo, a digital pulse with a duration of 900 microseconds will send the servo arm to a position of 0º, while on the other end, a pulse of 2100 microseconds will send the servo 180º in the other direction. The drawing in Figure 9-5 illustrates the typical movement arc of our Hitec servo. Every brand of servo will be different, with typical ranges of movement exceeding 180º, and some servos even spin in the opposite direction, so